DOCUMENTS

DOCUMENTS
Volltextindizierung

DOCUMENTS 5.0

© Copyright 2016 otris software AG. Alle Rechte vorbehalten.

Weitergabe und Vervielfältigung dieser Publikation oder von Teilen daraus sind, zu welchem Zweck und in welcher Form auch immer, ohne die ausdrückliche schriftliche Genehmigung durch die otris software AG nicht gestattet. In dieser Publikation enthaltene Informationen können ohne vorherige Ankündigung geändert werden.

Alle in dieser Publikation aufgeführten Wort- und Bildmarken sind Eigentum der entsprechenden Hersteller.

Änderungen in der Software sind vorbehalten. Die in diesem Handbuch enthaltenen Informationen stellen keinerlei Verpflichtung seitens des Verkäufers dar.

Dokumentenversionen			
Version	**Datum**	**Autor**	**Bemerkungen / Änderungen**
0.7	22.09.2014	Dittberner	Entwurf
0.9	04.07.2016	Dittberner	Umstellung auf DOCUMENTS 5
1.0	16.09.2016	Dittberner	DOCUMENTS 5.0b MySQL-Optimierung

Inhaltsverzeichnis

1. Einführung 4

1.1 Indextypen allgemein 4
1.2 Indextypen in DOCUMENTS 4
1.3 Die Stoppwortliste 4
1.4 Mustersuche und ihre Effizienz 4

2. Aktivierung des Volltextindexes 6

2.1 Auswahl der zu indizierenden Daten 6
2.2 Indizierung von Bestandsdaten 7

3. Recherchen mit dem Volltextindex 9

3.1 Verwendung 9
3.2 Beispiele mit Vergleich der Indextypen 9

4. Ergänzende Einstellungen 11

4.1 Suchmethoden 11
4.2 Parameter des Volltextindizierers 12
4.3 Änderung der Stoppwortliste 12
4.4 Spezielle Datenbankoptimierung für MS SQL Server 12
4.5 Optimierung für MySQL 13

Abbildungsverzeichnis 16

1. Einführung

1.1 Indextypen allgemein

Texte lassen sich zum Zweck einer schnellen Recherche auf verschiedene Arten indizieren. Die einfachste Variante ist ein gewöhnlicher Datenbankindex für die gesamte Textspalte (Primärindex). Ein Volltextindex ist im Vergleich etwas komplexer im Aufbau, bietet jedoch eine leistungsfähigere Recherche. Der Hauptunterschied der beiden Indextypen ist, dass ein Volltextindex den Feldinhalt in einzelne Wörter zerlegt. Der Primärindex arbeitet nicht mit Wörtern, sondern behandelt jeden Text als unstrukturierte Zeichenfolge. Ein weiterer Unterschied ist, dass Volltextindizes in der Regel auch Satz- und Sonderzeichen erkennen und beseitigen. Die Suche nach einem einzelnen Komma oder einem Leerzeichen im Volltext ist folglich zwecklos.

1.2 Indextypen in DOCUMENTS

Alle Mappenfelder können in **DOCUMENTS** über einen Primärindex recherchiert werden. Ab **DOCUMENTS 5.0** besteht die Möglichkeit, bestimmte Felder zusätzlich in einen Volltextindex aufzunehmen.

Dateianhänge von Mappen indiziert **DOCUMENTS** ausschließlich als Volltext. Dabei kommen externe Hilfsprogramme zum Einsatz, um den Volltext aus unterschiedlichen Dateitypen zu extrahieren (siehe das Verzeichnis „`docfilter`“ im Installationspfad).

1.3 Die Stoppwortliste

Manche Wörter kommen in Texten so häufig vor, dass sie als Suchkriterium ungeeignet sind. Volltextindizes filtern diese Wörter regelmäßig, damit der Index nicht zu umfangreich wird und effizient bleibt. Eine Suche nach einem Stoppwort liefert beim **DOCUMENTS**-Volltextindex keine Treffer. Viele Systeme filtern Stoppwörter allerdings auch bei Suchanfragen. Sie reagieren, als wären die Wörter nicht eingegeben worden. In **DOCUMENTS** kann dieses Verhalten – abhängig von der eingesetzten Software – bei Archivrecherchen auftreten.

Die Anpassung der **DOCUMENTS**-Stoppwortliste ist das Thema von Abschnitt 4.3.

1.4 Mustersuche und ihre Effizienz

Prinzipiell kann bei beiden Indextypen das Sonderzeichen „*“ als Platzhalter für eine beliebige Zeichenfolge benutzt werden. Auf diese Weise kann im Primärindex nach Teilen eines Feldwerts sowie im Volltextindex nach Teilen eines Worts gesucht werden. Der Zeitaufwand der Suche steigt erheblich an, wenn das Platzhalterzeichen weit vorne im Suchbegriff steht. Wenn es ganz vorne steht, muss die Datenbank den kompletten Index durchlesen. In diesem Fall ist der Index nahezu nutzlos.

Die Suche in einem Volltextindex ist vor allem deshalb schneller, weil jedes einzelne Wort ohne vorangestelltes Platzhalterzeichen gefunden werden kann.

Beispiele für einige Recherchen mit und ohne Platzhalter folgen im Abschnitt 3.2 (Seite 9). **DOCUMENTS** kann Platzhalterzeichen auch automatisch anfügen. Dies wird im Abschnitt 4.1 näher erläutert.

2. Aktivierung des Volltextindexes

2.1 Auswahl der zu indizierenden Daten

DOCUMENTS unterscheidet bei der Volltextindizierung zwischen Dateianhängen (Anlagen) und Mappenfeldern. Die direkt in der Mappe gespeicherten Daten (Titel, Erstelldatum u.s.w.) und die Namen von Anlagen werden nicht volltextindiziert. Die Indizierung von Anlagen wird über den Mappentyp gesteuert.

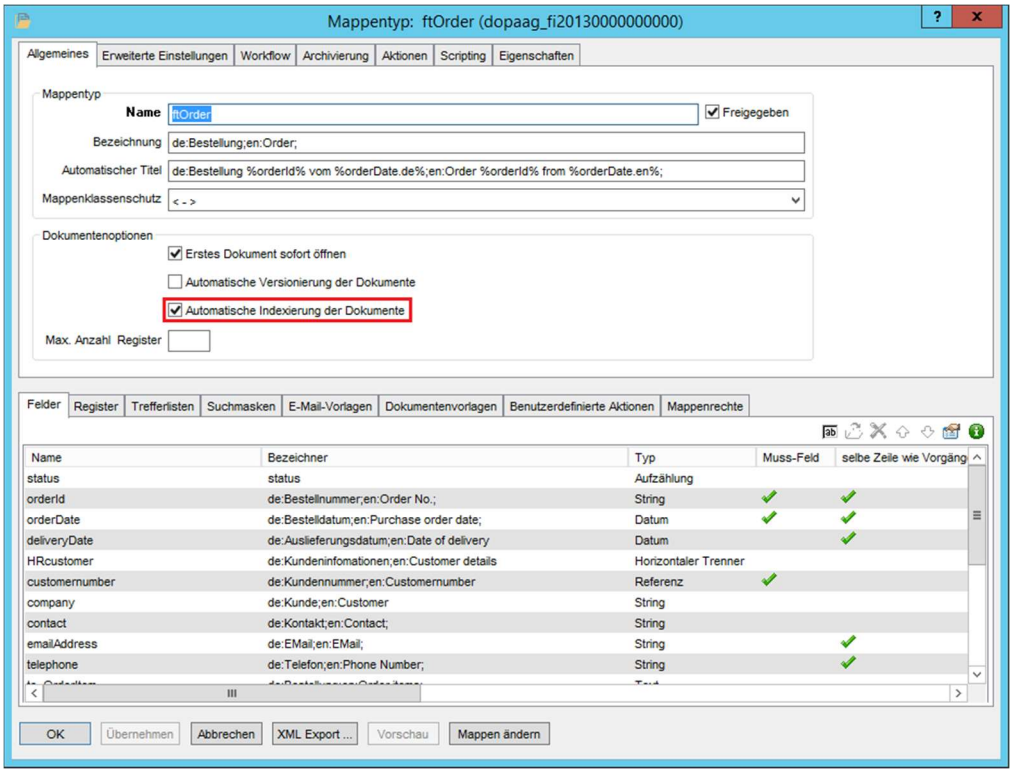

Abb. 1: Indexierung von Dokumenten am Mappentyp aktivieren

Für die Indizierung von Feldern gibt es eine entsprechende Checkbox im Felddialog.

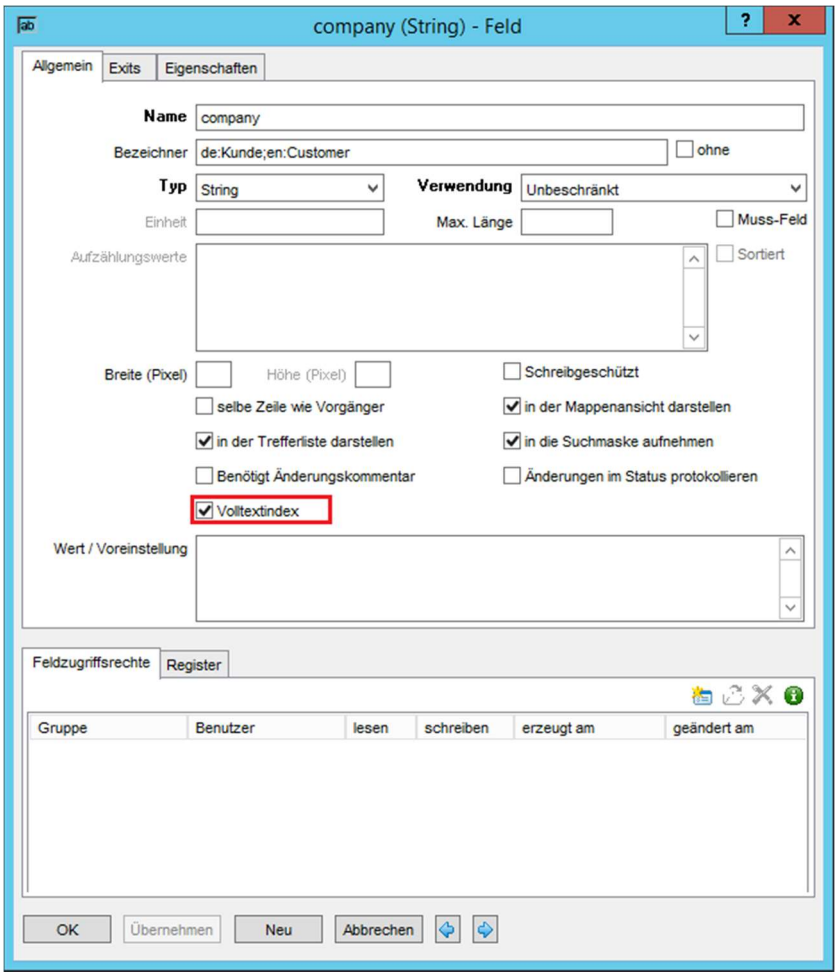

Abb. 2: Volltextindex für Felder aktivieren

Diese Einstellungen betreffen nur **DOCUMENTS**-Mappen *(„aktive Vorgänge“).* Für Archivmappen gelten systemspezifische Einstellungen, die der jeweiligen Dokumentation zu entnehmen sind.

Bei der Neudefinition eines Felds schlägt der **DOCUMENTS-Manager** die Volltextoption für Textfelder automatisch vor. Er entfernt sie bei anderen Feldtypen. Diese Vorauswahl ist nicht bindend. Es ist sinnvoll, von Fall zu Fall zu entscheiden, ob der Volltextindex gebraucht wird. Je weniger Daten indiziert werden müssen, desto schneller erfolgt das Speichern von Mappenänderungen.

2.2 Indizierung von Bestandsdaten

Für bestehende Mappen funktioniert der Volltextindex nicht unmittelbar. Es muss zunächst die Wartungsoperation „`reindex`“ ausgeführt werden. Wenn nur die Anlagen oder nur die Feldinhalte neu indiziert werden sollen, kann man „`reindex docs`“ bzw. „`reindex fields`“ angeben, um den Vorgang abzukürzen. Die Laufzeit dieser Operation kann im Stundenbereich liegen. Daher ist es vorteilhaft, zunächst alle gewünschten Einstellungen vorzunehmen, und die Neuindizierung erst zum Schluss auszulösen.

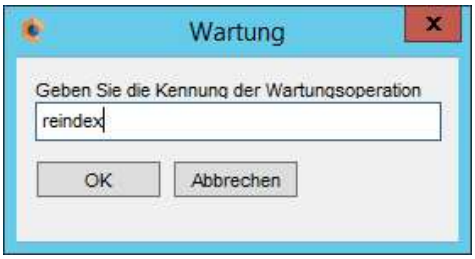

Abb. 3: Wartungsoperation "reindex"

Wichtiger Hinweis

Wenn vor Abschluss der Neuindizierung in dem System eine Suche läuft, die den Volltextindex benutzt, können Treffer ausbleiben. Bei Abfragen mit „enthält nicht“ können auch falsche Treffer entstehen. In ***sehr*** *speziellen Fällen kann das zu Folgefehlern in Portalskripten oder in externen Anwendungen führen. Ein extremes Beispiel wäre ein Skript, das mit einem „enthält nicht“-Filterausdruck falsche Mappen ermittelt, die es anschließend ohne weitere Sicherheitsabfragen löscht.*

*Als Gegenmaßnahmen bieten sich die Wartungskommandos „****jobs off****“ (vorher) und „****jobs on****“ (nachher) an sowie ein zeitweiliger Stopp des* DOCUMENTS-SOAP-Proxys*. Zur Sicherheit sollte auch der Web-Zugang gesperrt werden.*

3. Recherchen mit dem Volltextindex

3.1 Verwendung

Die Recherchefunktionen von **DOCUMENTS** benutzen den internen Volltextindex in folgenden Fällen.

1. Suche über die Weboberfläche bzw. mit dem Webrechercheoperator „*enthält*“ in einem volltextindizierten Feld. Davon ausgenommen sind Feldtypen mit numerischem oder binärem Charakter, bei denen eine Volltextrecherche zwecklos ist (z.B. Checkbox, Zeitstempel).

2. Suche in einem allgemeinen Volltextsuchfeld. Diese Suche schließt grundsätzlich alle Daten einer Mappe ein: Attribute wie z.B. Titel, Erstelldatum, sowie Feldinhalte, Dateinamen und Dateiinhalte. Innerhalb von Portalskripten und externen Anwendungen kann diese Form der Recherche mit dem Bezeichner „`Search_Fulltext`“ anstelle eines Feldnamens durchgeführt werden.

3.2 Beispiele mit Vergleich der Indextypen

In einem Textfeld einer Mappe sei der folgende, aus Abschnitt 1.4 übernommene Beispielsatz gespeichert.

Prinzipiell kann bei beiden Indextypen das Sonderzeichen „“ als Platzhalter für eine beliebige Zeichenfolge benutzt werden.*

Die folgende Tabelle zeigt, ob die beiden Indextypen für verschiedene Suchausdrücke einen Treffer erzeugen und wie der Zeitaufwand der Recherche einzuschätzen ist. Dabei wird vorausgesetzt, dass **DOCUMENTS** keine Platzhalterzeichen automatisch ergänzt, d.h. alle Suchmethodeneinstellungen haben den Wert 1 (exakte Suche).

Ausdruck	Indextyp	Treffer	Suchzeit Einzelfeld*	Suchzeit Volltextfeld*	Bemerkungen
Prinzipiell	Primär	Nein	unter 1s	unter 1s	Werte werden exakt verglichen, daher kein Treffer
	Voll	Ja	unter 1s	unter 1s	
Prinzipiell*	Primär	Ja	unter 1s	unter 1s	
	Voll	Ja	unter 1s	unter 1s	
P*	Primär	Ja	unter 1s	1s bis 2 s	
	Voll	Ja	unter 1s	1s bis 2 s	
Indextypen*	Primär	Nein	unter 1s	unter 1s	Gesuchter Ausdruck steht nicht am Feldanfang
	Voll	Ja	unter 1s	unter 1s	
Z*	Primär	Nein	unter 1s	unter 1s	
	Voll	Ja	unter 1s	unter 1s	Treffer beim Wort ‚Zeichenfolge'
ka*	Voll	Nein	unter 1s	unter 1s	„kann" würde passen, doch es ist ein Stoppwort und steht nicht im Index
!Prinzipiell	Primär	Ja	unter 1s	unter 1s	Wirkt wie „ungleich". Mit „*" dahinter ist es kein Treffer mehr
	Voll	Nein	unter 1s	unter 1s **	Wirkt wie „enthält nicht". Vorsicht bei Verwendung in Skripten! (siehe Hinweis unter 2.2)
Zeichenfolge	Primär	Ja	2s bis 3s	9s	
	Voll	Ja	3s	9s	
Zeichenfolge	Primär	Ja	3s	33s	
(jeweils nach Neustart aller Dienste)	Voll	Ja	3s	33s	
"*Zeichenfolge benutzt*" (in einer Zeile)	Primär	Ja	3s	10s	Dieses Ersatzmuster für eine Phrasensuche funktioniert nur, wenn im Feld genau ein Leerzeichen zwischen den Wörtern steht. Bei Mehreren oder Zeilenumbruch ist es kein Treffer.
"Zeichenfolge benutzt" (in einer Zeile)	Voll	Ja	unter 1s	unter 1s	Phrasensuche mit Volltextindex. Je mehr Wörter in der Phrase desto langsamer wird die Suche.

Testsystem: 16 GB RAM, Datenbank SQL Server 2012 64 Bit, 111663 aktive Mappen im System, darunter 61412 vom Testmappentyp; Feldanzahl gesamt: 2.521.727; Einzelwörter im Feld-Volltextindex: 8.338.279. Keine Volltextindizierten Anlagen vorhanden. Kein Mappenklassenschutz; Ordnerrechte deaktiviert. Jobs deaktiviert. Nur ein angemeldeter Benutzer im System.

Die Zeitangaben beziehen sich auf das reine Ermitteln der Mappen-Ids. Die Zeit für den Aufbau der ersten Trefferseite kommt jeweils noch hinzu.

Abb. 4: Unterschiede der beiden Indextypen bei der Suche

4. Ergänzende Einstellungen

4.1 Suchmethoden

Der Parameter „Suchmethode“ in den **Documents**-Einstellungen dient dazu, einen Suchbegriff automatisch mit Platzhalterzeichen einzuschließen, wenn eine Mustersuche für das Feld sinnvoll ist (z.B. nicht bei Boolean). Somit müssen die Benutzer nicht mehr ständig „*“ für ein bestimmtes Suchmuster eintippen. Von den drei Basismethoden sind beim Volltextindex nur zwei praktikabel. Die dritte Methode eignet sich aufgrund mangelnder Performance nicht. Die Methode 0 wird beim Volltextindex deshalb wie 2 behandelt. Es gibt separate Suchmethodeneinstellungen für **DOCUMENTS**-Mappen und für Archivmappen.

Suchmethoden in **DOCUMENTS**				
Nummer	Beschreibung	Benutzereingabe (Beispiel)	Suchmuster im Primärindex	Suchmuster im Volltextindex **
0	beliebiger Feldteil	foo	*foo*	foo*
1	exakte Suche	foo	foo	foo
2	Feldanfang / Wortanfang	foo	foo*	foo*
3*	beliebiger Feldteil; bei Volltextsuche vorangestellte Platzhalter unterdrücken	foo	*foo*	foo*
		*foo	*foo*	foo*
4*	exakte Suche; bei Volltextsuche vorangestellte Platzhalter unterdrücken	foo	foo	foo
		*foo	*foo	foo
5*	Feldanfang / Wortanfang; bei Volltextsuche vorangestellte Platzhalter unterdrücken	foo	foo*	foo*
*: Die Zusatzmethoden 3 bis 5 werden nur für aktive Mappen unterstützt. Sie sollten nicht als Archivsuchmethode angegeben werden. **: Spezielle Einstellung für EE.i für EE.x erforderlich; siehe Text.				

Abb. 5: Suchmethoden in DOCUMENTS

Wenn **DOCUMENTS** mit einem **EE.i** oder **EE.x**-Archiv verbunden ist, wird zwecks Abwärtskompatibilität mit der bisherigen Einstellung noch kein Suchmuster für Volltextanfragen erzeugt. Dazu muss in den **DOCUMENTS**-Einstellungen erst die Eigenschaft „`FulltextMethod`“ mit der Methodennummer 2 als Wert eingetragen werden. Sie gilt dann für alle Volltextrecherchen systemübergreifend. Umgekehrt bewirkt diese Eigenschaft mit dem Wert 1, dass beim Einsatz des **DOCUMENTS-Archivs (EAS)** die Volltextrecherche wortgenau erfolgt, solange kein Platzhalterzeichen manuell angegeben wird. Somit kann die Suchmethode für Primärindex und Volltextindex unabhängig gesteuert werden.

4.2 Parameter des Volltextindizierers

Im Unterverzeichnis „`docfilter`“ der **DOCUMENTS**-Installation befindet sich das Hilfsprogamm „`wpl`“, das **DOCUMENTS** zum Indizieren von Anlagen einer Mappe benutzt. In der zugehörigen Konfigurationsdatei „`wpl.ini`“ gibt es folgende Einstellungen.

- „`wordMode`“ legt fest, welche Zeichen als Teil eines Worts akzeptiert werden. Alle Übrigen gelten in der Regel als Trennzeichen. Im Modus 0 werden nur reine Buchstabenfolgen akzeptiert. Im Modus 1 sind auch Kombinationen aus Buchstaben und Zahlen erlaubt. Modus 2 erlaubt folgende Zeichen als Teil eines Worts: Punkt, Komma, Semikolon, Schrägstrich, Minuszeichen und At-Zeichen. Modus 3 kombiniert die Erweiterungen der Modi 1 und 2.
- „`minWordLength`“ legt fest, aus wie vielen Zeichen ein Wort mindestens bestehen muss, um im Volltextindex gespeichert zu werden. Die Voreinstellung ist 3.
- Die Einträge in dem Abschnitt [filter] bestimmen, welches Kommandozeilentool zum Extrahieren der Textinformation aus einem bestimmten Dateityp benutzt werden soll. Die Einträge entsprechen folgendem Muster.

```
Dateierweiterung=Kommandozeile
```

Der Platzhalter *„%d“* steht hierbei für den Dateinamen inklusive Pfad.

4.3 Änderung der Stoppwortliste

Bei der Datei „`wpl.swl`“ im Verzeichnis „`docfilter`“ handelt es sich um die vorinstallierte Stoppwortliste. Sie kann nach Bedarf mit einem gewöhnlichen Texteditor bearbeitet werden. In diesem Fall sollte zuerst eine Sicherungskopie von der Originaldatei erstellt werden und später eine von der individuellen Version. Ferner darf die Zeichenkodierung der Stoppwortliste nicht verändert werden.

Bei einem Update von **DOCUMENTS** wird die Stoppwortliste eventuell überschrieben. Ein Vergleich der alten Originalliste mit der frisch installierten Version und mit der gesicherten individuellen Version erleichtert dann das Zusammenführen von Änderungen aus dem Update mit eigenen Anpassungen.

4.4 Spezielle Datenbankoptimierung für MS SQL Server

Zugunsten der Portierbarkeit verwendet **DOCUMENTS** für den internen Volltextindex keine Sonderfunktionen einer Datenbanksoftware, sondern gewöhnliche Datenbanktabellen. Pro **DOCUMENTS**-Mandant werden zwei Tabellen gleicher Struktur erzeugt (DlcDocumentI… für Anlagen und DlcFTII… für Felder). Zu diesen Tabellen gehört jeweils ein Datenbankindex „<Tabellenname>_word“.

Es hat sich herausgestellt, dass beim **Microsoft SQL Server** eine Vereinfachung der Ausführungspläne für die Volltextsuche erreicht werden kann, wenn dieser Index etwas erweitert wird. Die Erweiterung belegt ungefähr 80 Bytes pro Datensatz. In Produktivsystemen mit Millionen Indexeinträgen macht das allerdings hunderte Megabytes aus. Daher sollte die Erweiterung nur nach Abwägung der Vor- und Nachteile benutzt werden.

Der **DocumentsServer** verwaltet diesen Index selbst und nimmt nach jedem Start die datenbankseitige Anpassung vor, wenn die Konfiguration geändert wurde. Um die Erweiterung zu aktivieren, genügt folgende Ergänzung in der „`documents.ini`“:

```
$FTI_LARGE 1
```

Grundsätzlich funktioniert die Indexerweiterung auch mit **Oracle DB** und **MySQL**, jedoch konnte hier bisher keine Verbesserung der Geschwindigkeit festgestellt werden.

4.5 Optimierung für MySQL

In der Standardkonfiguration kann die Suche nach volltextindizierten Feldinhalten mit **MySQL 5.5** sehr langsam werden, wenn der Index für ein gesuchtes Wort mehrere tausend Einzeltreffer liefert. Das Problem ist, dass dieses Datenbanksystem noch keine „Hashtable-Joins“ (Tabellenverknüpfungen per Prüfsummenliste) unterstützt.

Ein zweiter kritischer Punkt ist, dass die Datenbank bei der Kombination der Abfrageelemente „Subquery“ und „Left outer join“ erfahrungsgemäß die Subquery zuletzt verarbeitet, auch wenn sie keine mit der anderen Verknüpfung ermittelten Spalten benötigt. Das ist ungünstig, weil auf diese Weise manchmal viele Tabellenzeilen aus der rechten Tabelle der Verknüpfung unnötig gelesen werden. Die Subquery kann nämlich viele Zeilen aus dem Zwischenergebnis wieder eliminieren. In **DOCUMENTS** kann dies zum Problem werden, wenn im Volltextindex gesucht und zugleich nach einem Feld sortiert werden soll. Eine Sortierung nach Attributen (Titel, Änderungsdatum, usw.) ist hier unproblematisch.

Neuere Versionen des Datenbanksystems haben diese Einschränkungen möglicherweise nicht mehr – Änderungen an dieser Stelle wären dann nicht mehr notwendig. Um das Problem anderweitig zu entschärfen kann **DOCUMENTS** ab Version **5.0b** die Recherche wahlweise mit temporären Datenbanktabellen durchführen. Die Anwendung erstellt hierbei selbst die notwendigen Hilfstabellen, die eine schnellere Verknüpfung bei großer Trefferanzahl ermöglichen. Sobald das Suchergebnis feststeht, werden diese Tabellen unverzüglich wieder gelöscht.

Dieses Verfahren setzt allerdings voraus, dass dem **DocumentsServer** datenbankseitig das Recht zum Erstellen und Löschen solcher Tabellen gewährt wird. Möglicherweise muss auch die MySQL-Systemvariable „`max_heap_table_size`“ angepasst werden, um ausreichend Arbeitsspeicher für diese Tabellen zu reservieren.

Es gibt Anwendungsfälle, in denen die temporären Tabellen die Recherche verlangsamen. Wenn beispielsweise nach einem Feld „Rechnungsdatum“ (Primärindex) und einem Feld „Rechnungstext“ (Volltextindex) gefiltert wird, werden intern immer alle Treffer für das Volltextfeld ermittelt, auch wenn es zu dem Datum nur 3 Rechnungen gibt. In diesem Fall wären die herkömmlichen Datenbankabfragen schneller. **DOCUMENTS** kann das nicht für jeden Einzelfall ermitteln. Stattdessen wurde die Möglichkeit geschaffen, die temporären Tabellen nur zu verwenden, wenn kein Primärindexfeld an der Suche beteiligt ist.

In den **Documents**-Einstellungen steuert die Eigenschaft „`TempTabSearchPolicy`", wann temporäre Tabellen bei Suchanfragen benutzt werden sollen. Als Wert ist eine ganze Zahl

anzugeben. Die nachfolgende Tabelle listet mögliche Werte und deren Auswirkung auf. Um mehrere Optionen zu kombinieren, müssen die Tabellenwerte addiert werden.

Eigenschaftswert TempTableSearchPolicy	**Funktion**
1	Eine temporäre Tabelle wird erzeugt, wenn die Suchanfrage den Volltextindex für Felder benutzt, jedoch nicht den Primärindex.
2	Eine temporäre Tabelle wird erzeugt, wenn die Suchanfrage sowohl den Volltextindex für Felder benutzt, als auch den Primärindex.
4	Zur datenbankseitigen Sortierung des Suchergebnisses wird vorab eine temporäre Tabelle erzeugt, wenn auf diese Weise die beiden SQL-Operationen „subquery“ und „left outer join“ voneinander entkoppelt werden können.

Alle derzeit eingesetzten temporären Tabellen enthalten nur eine einzige Spalte sowie einen eindeutigen Index für diese Spalte. Die mit den Optionen 1 oder 2 erzeugten Tabellen enthalten standardmäßig den Spaltentyp „VARCHAR(40)“, was **MySQL** bei Tabellen im Arbeitsspeicher allerdings mit „CHAR(40)“ gleichsetzt. Für jedes im Feld-Volltextindex gefundene Wort wird eine Zeile in diese Tabelle geschrieben. Wird nach zwei Wörtern gesucht, dann entstehen auch zwei Tabellen, außer es handelt sich um eine Phrasensuche. Wenn mehrere Benutzer gleichzeitig recherchieren, vervielfacht sich der Platzbedarf entsprechend.

Die mit dem Eigenschaftswert 4 erzeugten Tabellen enthalten lediglich eine INTEGER-Spalte (32 Bit) und pro gefundener Mappe eine einzige Zeile. Sie sind hinsichtlich des Speicherbedarfs also weit weniger kritisch.

Falls der Speicherbedarf der temporären Tabellen ausufert, gibt es eine Möglichkeit, auch die Tabellen der Optionen 1 und 2 auf eine INTEGER-Spalte zu reduzieren. Dies hat einen moderaten Geschwindigkeitsverlust zur Folge (ca. 2 Sekunden je hunderttausend Volltext-treffer). Für diese Speicherersparnis benötigt die Anwendung allerdings eine zusätzliche persistente Datenbanktabelle „LeanFileIdMap“, deren Inhalt sich aus Daten der Tabelle „DlcFile“ ableitet und über Trigger aktuell gehalten werden muss. Auf folgende Weise kann die Tabelle eingerichtet werden.

- Beenden Sie den **DocumentsServer**.
- Starten Sie eine Eingabeaufforderung und Wechseln Sie in das Verzeichnis des **DocumentsServers**. Dort sollte ein SQL-Skript „mysql_fileidmap_create.sql“ liegen. Falls nicht, fordern Sie es bei der **otris software AG** an.

- Starten Sie von hier den MySQL-Konsolenclient. Verbinden Sie sich direkt mit der **DOCUMENTS**-Datenbank.
- Führen Sie mit dem „source“-Kommando das Skript aus.
- Falls Fehlermeldungen angezeigt werden, prüfen Sie, ob Sie mit dem korrekten Schema arbeiten („USE DATABASE ...“) und ob Ihnen möglicherweise Berechtigungen fehlen.

Mit dem Skript „mysql_fileidmap_delete.sql“ kann die Änderung bei Bedarf wieder rückgängig gemacht werden.

Abbildungsverzeichnis

Abb. 1: Indexierung von Dokumenten am Mappentyp aktivieren 6
Abb. 2: Volltextindex für Felder aktivieren 7
Abb. 3: Wartungsoperation "reindex" 8
Abb. 4: Unterschiede der beiden Indextypen bei der Suche 10
Abb. 5: Suchmethoden in DOCUMENTS 11